

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: MLA0303

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks. (BL2, BL3, BL6).*

Assessment Tool Weightage: 25%

Dr DUMPALA PRASANTH

Code of Conduct:

I **Charandeep N C (192472196)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Charandeep N.C

QUESTIONS: 10

Total Marks: 50

Annexure A

Data Interpretation

Duration: 1hr

1. A Reinforcement Learning agent is trained for multiple episodes and the reward values increase gradually from 10 to 45. Interpret the trend in rewards and defend what indicates about the learning performance and policy improvement of the agent. (5 Marks)

Answer:

1. Interpretation of the Data

The reward values rise steadily and monotonically from 10 in the earliest episodes to 45 in the later episodes. There are no large drops or plateaus reported, which means the trend is a smooth, positive learning curve rather than random noise.

This is a large improvement: the reward more than quadruples over training, which is a strong, unambiguous behavioural signal rather than a marginal or statistically weak change.

2. Analysis - Why the Trend Occurs

In every episode the agent observes states, takes actions, receives rewards, and updates its internal value estimates (e.g., $V(s)$ or $Q(s,a)$) using a Bellman-based update such as $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$.

Early in training these estimates are inaccurate (often initialised at zero or randomly), so the agent's actions are close to random and rewards are low (~ 10). As more episodes are experienced, the temporal-difference error $[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ shrinks toward zero, meaning the value estimates increasingly match the true expected return, and the greedy action selected in each state becomes closer to the truly optimal action.

This is precisely why the observed reward keeps climbing: better value estimates translate directly into better action choices and hence higher accumulated reward per episode.

3. Application of RL Concept - Policy Improvement

The Policy Improvement Theorem states that if a policy is updated greedily with respect to an improving value function, the new policy can never perform worse than the old one. The steadily rising curve is the empirical signature of this theorem in action.

A smooth (non-oscillating) rise additionally suggests the exploration rate and learning rate are well tuned - excessive exploration or too high a learning rate would show up as a noisy, jagged curve instead of the clean upward trend given here.

4. Calculation (using the exact figures given in the question)

Given: initial reward = 10, final reward = 45.

Absolute increase = $45 - 10 = 35$

Percentage increase = $(35 / 10) \times 100 = 350\%$

This 350% gain, measured directly from the data provided, quantitatively confirms a large and sustained improvement in performance, not a marginal change.

5. Conclusion

The rising reward trend is strong evidence of genuine learning and continuous policy improvement. It shows the agent's value function is converging toward the true value function and that its policy is converging toward the optimal policy π^* . Formal confirmation of full convergence would require observing the curve until it plateaus; until then, the 350% rise is a clear, positive indicator of effective learning performance.

2. An epsilon-greedy agent is tested with different epsilon values such as 0.1, 0.3 and 0.5. The cumulative reward obtained is highest when epsilon is 0.1 and lowest when epsilon is 0.5. Justify and apply the concept of exploration vs exploitation to interpret why the reward changes with epsilon and identify which epsilon value provides better performance.

(5 Marks)

Answer:

1. Interpretation of the Data

Cumulative reward is highest at $\epsilon = 0.1$, intermediate at $\epsilon = 0.3$, and lowest at $\epsilon = 0.5$. Reward therefore falls as ϵ rises - a clear inverse relationship between the exploration rate and cumulative performance in this task.

2. Analysis - The Exploration-Exploitation Trade-off

In an ϵ -greedy policy, the agent exploits its current best-known action with probability $(1-\epsilon)$ and explores a uniformly random action with probability ϵ . Exploitation uses accumulated knowledge to earn reward now, whereas exploration sacrifices some reward in order to gather information about actions that might be even better.

When ϵ is small (0.1), the agent spends the vast majority of its decisions exploiting the best action found so far, so once the Q-values are reasonably accurate it consistently earns near-optimal reward.

When ϵ is large (0.5), half of all decisions are random, so even a perfectly learned policy is overridden by a random action half the time - this directly caps the achievable cumulative reward regardless of how good the learned values are.

3. Application of RL Concept

This matches the classic exploration-exploitation dilemma central to reinforcement learning: too little exploration risks getting stuck on a suboptimal action (never discovering a better one), while too much exploration wastes reward on actions already known to be inferior.

$\epsilon = 0.1$ sits closer to the 'mostly exploit, occasionally explore' end of the spectrum, which is generally preferred once the agent has had enough episodes to form reliable value estimates. A common refinement is an annealed/decaying ϵ (high early for exploration, decayed toward a low value such as 0.1 later for exploitation), which would be expected to outperform any single fixed ϵ tested here.

4. Calculation (using the exact epsilon values given in the question)

The fraction of decisions spent exploiting the current best action is $(1 - \epsilon)$, and the fraction spent exploring at random is ϵ :

$\epsilon = 0.1 \rightarrow$ Exploitation = $(1 - 0.1) \times 100 = 90\%$, Exploration = 10%

$\epsilon = 0.3 \rightarrow$ Exploitation = $(1 - 0.3) \times 100 = 70\%$, Exploration = 30%

$\epsilon = 0.5 \rightarrow$ Exploitation = $(1 - 0.5) \times 100 = 50\%$, Exploration = 50%

Because exploitation always selects the highest-value action while exploration selects randomly among all actions (whose average value is lower than the best), a higher exploitation percentage directly produces a higher expected reward. This computed ranking, $90\% > 70\% > 50\%$, exactly matches the given performance order: $\text{reward}(\epsilon=0.1) > \text{reward}(\epsilon=0.3) > \text{reward}(\epsilon=0.5)$.

5. Conclusion

$\epsilon = 0.1$ provides the better performance in this scenario because it maximises exploitation of the learned policy (90% of decisions) while retaining just enough exploration (10%) to avoid stagnation. For deployment or evaluation phases after learning has stabilised, an even lower or decaying ϵ would typically be recommended.

3. Two Reinforcement Learning algorithms, Q-Learning and SARSA, are compared based on average rewards obtained over 100, 200 and 300 episodes. Q-Learning consistently shows slightly higher reward values than SARSA. Identify the performance difference between these algorithms and determine which algorithm converges faster based on the observed results. (5 Marks)

Answer:

1. Interpretation of the Data

Across all three checkpoints (100, 200 and 300 episodes), Q-Learning's average reward is consistently slightly higher than SARSA's. The gap is present at every checkpoint, indicating a stable, systematic difference rather than a one-off fluctuation. The question does not provide exact reward figures, so the comparison below is derived directly from the two algorithms' update rules rather than assumed numbers.

2. Analysis - Off-Policy vs On-Policy Updates

Q-Learning is off-policy: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$. It bootstraps using the maximum Q-value achievable from the next state, i.e., it evaluates the greedy/optimal policy regardless of which action the exploring agent actually takes next.

SARSA is on-policy: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$, where a' is the action actually chosen by the current (ϵ -greedy, still-exploring) policy. Because occasional exploratory/random actions are folded into the update, SARSA's value estimates - and the rewards it earns while learning - are systematically more conservative.

3. Application of RL Concept - Convergence Speed

Because Q-Learning always bootstraps toward the maximum future value, it pushes its estimates toward the optimal action-value function Q^* more directly and aggressively - this $\max_{a'}$ operator is exactly what produces a higher reward estimate at every checkpoint (100, 200, 300 episodes) compared to SARSA's $Q(s',a')$ term, which is bounded by whatever action the exploring policy actually selects (sometimes a non-optimal, exploratory one).

SARSA converges to the value of the policy actually being followed (including its exploration), which is safer in risky environments (e.g., a cliff-walking task) but slower to reach the same reward level as Q-Learning here.

4. Calculation / Formal Comparison (based on the update rules, since no reward figures are given)

Q-Learning target $= r + \gamma \max_{a'} Q(s',a')$

SARSA target $= r + \gamma Q(s', a')$ where a' is the actually-chosen (possibly exploratory) action

Since $\max_{a'} Q(s',a') \geq Q(s',a')$ for any specific a' (the max over all actions is always at least as large as the value of one particular action), the Q-Learning target is algebraically guaranteed to be $\geq$ the SARSA target whenever the policy is still exploring. This inequality is the mathematical reason Q-Learning's average reward is consistently at or above SARSA's at 100, 200 and 300 episodes.

5. Conclusion

Q-Learning converges faster and yields a higher reward in this comparison because its off-policy, max-based bootstrapping targets the optimal policy directly, and this is provably $\geq$ SARSA's on-policy target whenever exploration is still occurring. The trade-off is that Q-Learning can be more prone to overestimation bias and riskier behaviour, so SARSA remains preferable in safety-critical or highly stochastic settings despite its slightly slower convergence.

4. An RL agent is trained using different discount factor values such as 0.2, 0.5 and 0.9. The average reward increases as the discount factor increases. Discuss how the discount factor affects long-term reward optimization and justify which discount factor is most suitable for achieving better learning performance.

(5 Marks)

Answer:

1. Interpretation of the Data

Average reward increases monotonically as the discount factor γ rises from 0.2 to 0.5 to 0.9. This shows a clear positive relationship between how far into the future the agent 'looks' and the reward it ultimately achieves in this task.

2. Analysis - Role of the Discount Factor

The return the agent tries to maximise is $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots$. γ controls how quickly the weight given to future rewards decays.

With $\gamma = 0.2$ (low), future rewards are almost ignored, making the agent myopic and unable to plan multi-step strategies that trade a small immediate loss for a larger later gain.

With $\gamma = 0.9$ (high), the weight decays much more slowly, so rewards many steps away still meaningfully influence the current decision, letting the agent plan over a long horizon.

3. Application of RL Concept - Long-Term Optimisation

This directly explains the observed data: tasks that require multi-step planning are optimised far better when γ is close to 1, because the agent is effectively being told to care about long-term cumulative success, not just the next reward.

γ must nonetheless remain strictly less than 1 to keep the infinite-horizon return mathematically bounded/finite and to keep Bellman updates numerically stable.

4. Calculation (using the exact gamma values given in the question)

Weight on a reward k steps ahead = γ^k . Computing this for $k = 1, 2, 3$ with each of the three given values of γ :

$\gamma = 0.2$: $\gamma^1 = 0.200$, $\gamma^2 = 0.040$, $\gamma^3 = 0.008$ (future reward vanishes almost immediately)

$\gamma = 0.5$: $\gamma^1 = 0.500$, $\gamma^2 = 0.250$, $\gamma^3 = 0.125$

$\gamma = 0.9$: $\gamma^1 = 0.900$, $\gamma^2 = 0.810$, $\gamma^3 = 0.729$ (future reward remains substantial for many steps)

This calculation, done with the actual γ values supplied (0.2, 0.5, 0.9), shows the weight retained after 3 steps is 0.008 at $\gamma=0.2$ versus 0.729 at $\gamma=0.9$ - a 91x difference - which quantitatively explains why average reward rises sharply as γ increases.

5. Conclusion

$\gamma = 0.9$ is the most suitable discount factor for achieving better learning performance in this task because it allows the agent to properly value delayed rewards and plan over a long horizon, which both the observed increasing-reward trend and the γ^k calculation above confirm.

5. The success rate of an RL agent increases from 45 percent at 50 episodes to 92 percent at 200 episodes. Discuss the improvement in success rate and interpret whether the agent has learned an optimal policy. Provide reasoning based on the trend of the data. (5 Marks)

Answer:

1. Interpretation of the Data

Success rate climbs from 45% at 50 episodes to 92% at 200 episodes. This is a large, sustained improvement rather than a marginal change.

2. Analysis - What Rising Success Rate Reflects

Success rate is a direct behavioural measure of policy quality: it counts how often the agent's current policy actually achieves the task goal. Its steady rise means the agent is consistently discovering and reinforcing action sequences that lead to success, and is applying that knowledge more reliably episode after episode.

Curves of this shape (fast early gains, then typically slowing down) are characteristic of value-based learning approaching an asymptote, consistent with the value function converging.

3. Application of RL Concept - Optimality vs Near-Optimality

A high success rate is necessary but not sufficient evidence of a fully optimal policy. Optimality would require the curve to plateau (no further statistically significant improvement with more training) at the maximum achievable success rate for the environment.

At 92%, the agent has clearly learned a substantially improved, near-optimal policy, but without evidence of a plateau beyond 200 episodes we cannot conclusively claim full convergence to π^* .

4. Calculation (using the exact figures given in the question)

Given: success rate = 45% at 50 episodes, success rate = 92% at 200 episodes.

Increase in success rate = $92\% - 45\% = 47$ percentage points

Increase in episodes = $200 - 50 = 150$ episodes

Average rate of improvement = $47 / 150 \approx 0.313$ percentage points per episode

This computed rate (≈ 0.31 points/episode) is a directly usable, data-derived measure of how quickly the policy is improving, and can be compared against future checkpoints to detect whether the improvement rate is slowing (a sign of approaching convergence).

5. Conclusion

The data shows strong, consistent policy improvement (47-point gain, ≈ 0.31 points/episode) and a near-optimal policy at 200 episodes. Continued monitoring past 200 episodes, looking for the success-rate curve to flatten, is recommended before formally declaring the policy fully optimal.

6. An RL agent is trained in the CartPole environment with different learning rates (0.01, 0.05, 0.1). The agent achieves higher stability at 0.05 compared to the other values. Analyze how learning rate influences convergence and justify which value provides the best trade-off between speed and stability. (5 Marks)

Answer:

1. Interpretation of the Data

Among the three learning rates tested (0.01, 0.05, 0.1) on CartPole, the agent shows the highest stability at $\alpha = 0.05$, implying both the smallest and the largest tested rates under-perform on this metric relative to the middle value.

2. Analysis - Effect of Learning Rate on Updates

The learning rate α scales the size of every value update: $Q(s,a) \leftarrow Q(s,a) + \alpha \times [\text{target} - Q(s,a)]$. It is the single parameter most responsible for the trade-off between how quickly the agent adapts and how stable those adaptations are.

At $\alpha = 0.01$, each update is tiny, so the value function changes very slowly; this is stable in the sense of low variance, but it is also slow to converge and may still be far from the optimal values within a limited training budget.

At $\alpha = 0.1$, each update is large. In a continuously changing, dynamic environment like CartPole, large updates can cause the value estimates to overshoot and oscillate around the true values rather than settling - reducing stability even though early learning may look fast.

3. Application of RL Concept - Speed vs Stability Trade-off

$\alpha = 0.05$ sits between these extremes: it is large enough to make meaningful progress each episode while being small enough to avoid the overshoot/oscillation seen at 0.1. This is the classic trade-off for step sizes in stochastic approximation - too small under-fits, too large diverges/oscillates, and a well-chosen intermediate value balances both.

4. Calculation (using the exact learning-rate values given in the question)

The number of updates n required for the estimation error to halve follows $(1-\alpha)^n = 0.5$, i.e. $n = \ln(0.5) / \ln(1-\alpha)$. Computing this with each given α :

$\alpha = 0.01$: $n = \ln(0.5)/\ln(0.99) = -0.6931/-0.01005 \approx 68.9 \rightarrow \sim 69$ updates to halve the error

$\alpha = 0.05$: $n = \ln(0.5)/\ln(0.95) = -0.6931/-0.05129 \approx 13.5 \rightarrow \sim 14$ updates to halve the error

$\alpha = 0.1$: $n = \ln(0.5)/\ln(0.90) = -0.6931/-0.10536 \approx 6.6 \rightarrow \sim 7$ updates to halve the error

This shows $\alpha = 0.05$ halves the error about 5x faster than $\alpha = 0.01$, while needing only roughly double the updates of $\alpha = 0.1$ - i.e., it captures most of the speed advantage of a high learning rate while avoiding the overshoot risk that a step size twice as large introduces, which is exactly why it is the empirically most stable choice.

5. Conclusion

$\alpha = 0.05$ gives the best trade-off between convergence speed and stability for this CartPole setup and is the recommended learning rate, consistent with both the observed peak stability at that value and the update-count calculation above.

7. A Q-learning agent is tested with different exploration strategies: ϵ -greedy, softmax, and random exploration. The cumulative reward is highest with ϵ -greedy. Compare the strategies and defend why ϵ -greedy achieves better

performance in this scenario.

(5 Marks)

Answer:

1. Interpretation of the Data

Among ϵ -greedy, softmax, and random exploration, cumulative reward is highest under ϵ -greedy, meaning it outperformed both alternative exploration strategies in this experiment. No specific reward or probability values are given in the question, so the comparison below is derived from how each strategy allocates action choices.

2. Analysis - Comparing the Three Strategies

Random exploration ignores learned Q-values entirely and picks actions uniformly at random. It therefore continues to 'waste' a constant fraction of decisions on poor actions even after the agent has learned which actions are best, which caps its cumulative reward at a low level throughout training.

Softmax (Boltzmann) exploration selects actions with probability proportional to $\exp(Q(s,a)/\tau)$, so it does use learned values, weighting better actions more heavily. However, its behaviour is highly sensitive to the temperature τ : too high and it behaves almost like random exploration; too low and it collapses to greedy exploitation before the value estimates are reliable, risking premature convergence to a suboptimal action.

ϵ -greedy exploits the current best action with probability $(1-\epsilon)$ and explores uniformly with probability ϵ . It has only one simple, easy-to-tune parameter and gives a clean guarantee of continued exploration without the extra sensitivity that softmax's temperature introduces.

3. Application of RL Concept - Robustness of ϵ -greedy

In practice, softmax can match or beat ϵ -greedy when τ is carefully tuned, but ϵ -greedy tends to be more robust 'out of the box' because it does not require calibrating an extra hyperparameter, and it reliably exploits the best-known action most of the time once learning has progressed - explaining its superior cumulative reward in this test.

4. Formal Comparison (no numeric reward data given, so expressed generally)

Let $|A|$ be the number of available actions and ϵ the exploration rate under ϵ -greedy.

Random exploration: probability of choosing the best action on any given step = $1/|A|$ (equal chance for every action, regardless of learned value).

ϵ -greedy: probability of choosing the best action = $(1-\epsilon) + \epsilon/|A|$, which is always $\geq 1/|A|$ for any $\epsilon < 1$, and approaches 100% as $\epsilon \rightarrow 0$.

Softmax: probability of choosing the best action depends on $\exp(Q_{\text{best}}/\tau)$ relative to the other exponentials, which can equal, exceed, or fall short of ϵ -greedy's probability depending entirely on τ - i.e., its performance is not guaranteed and requires tuning, unlike ϵ -greedy's monotonic, predictable relationship with ϵ .

5. Conclusion

ϵ -greedy achieves the best cumulative reward here because it offers a simple, robust, and mathematically predictable balance between exploiting the learned policy and exploring alternatives, without the tuning sensitivity of softmax or the wastefulness of pure random exploration.

8. An RL agent trained in a maze environment initially takes 50 steps to reach the goal, but after 300 episodes it reduces to 15 steps. Interpret the reduction in steps and evaluate whether this indicates convergence to an optimal policy. (5 Marks)

Answer:

1. Interpretation of the Data

The number of steps needed to reach the goal in the maze falls from 50 (initial episodes) to 15 (after 300 episodes).

2. Analysis - Why Step Count Falls

Early on, the agent has no useful value estimates, so it wanders through the maze largely at random, taking many unnecessary steps (backtracking, dead ends) before stumbling onto the goal.

As episodes accumulate, Q-values (or the value function) are repeatedly updated toward the true expected return, which - assuming a per-step cost or discounting that favours shorter paths - makes shorter routes to the goal have higher value than longer ones. The greedy policy therefore increasingly favours direct paths, and the step count falls.

3. Application of RL Concept - Convergence to Optimality

Whether 15 steps represents full convergence to the optimal policy depends on the maze's true shortest-path length. If the actual minimum number of steps from start to goal is close to 15,

the policy has essentially converged to optimal. If a materially shorter path exists and the step count has not yet plateaued, the agent has improved substantially but has not fully converged.

4. Calculation (using the exact figures given in the question)

Given: initial steps = 50, steps after 300 episodes = 15.

Reduction = $50 - 15 = 35$ steps

Percentage improvement = $(35 / 50) \times 100 = 70\%$

A 70% efficiency gain, computed directly from the given data and sustained over 300 episodes (rather than a single lucky run), is a strong quantitative indicator of learned, repeatable path efficiency rather than chance.

5. Conclusion

The large, consistent 70% drop in steps-to-goal strongly suggests the agent is at or very near the optimal policy for this maze; formal confirmation would require verifying that step count no longer decreases significantly with additional training and comparing 15 against the maze's known shortest path length.

9. A SARSA agent is trained with discount factors of 0.4, 0.7, and 0.95. The agent achieves the highest long-term reward with 0.95. Discuss the role of discount factor in balancing immediate vs. future rewards and justify why 0.95 is most effective. (5 Marks)

Answer:

1. Interpretation of the Data

Testing SARSA with discount factors 0.4, 0.7 and 0.95 shows long-term reward rising with γ , peaking at $\gamma = 0.95$.

2. Analysis - Discount Factor in SARSA's On-Policy Update

SARSA updates using $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$, where $Q(s',a')$ reflects the value of the action actually taken by the current, exploring policy. γ still governs how much this future, on-policy value contributes to the present update.

At $\gamma = 0.4$, the agent weighs only near-term consequences heavily; it may greedily grab a small reward now while ignoring a much larger reward reachable a few steps later, capping long-term performance.

As γ rises to 0.7 and then 0.95, the agent's updates increasingly incorporate the value of future states, so it learns to sacrifice small immediate rewards for larger cumulative rewards down the line.

3. Application of RL Concept - Balancing Immediate vs Future Reward

This illustrates the general principle that γ trades off immediate gratification against long-term optimality. Because SARSA is on-policy, its future-value term already reflects the exploratory behaviour the agent actually exhibits, so a high γ here lets the agent properly credit long-horizon consequences of its real, exploration-inclusive behaviour.

4. Calculation (using the exact gamma values given in the question)

Weight retained after k steps = γ^k . Computing this with the three given values at $k = 5$ and $k = 20$:

$\gamma = 0.40$: $\gamma^5 = 0.40^5 = 0.01024$ ($\approx 1.0\%$) , $\gamma^{20} = 0.40^{20} \approx 0.0000000011$ ($\approx 0\%$)

$\gamma = 0.70$: $\gamma^5 = 0.70^5 = 0.16807$ ($\approx 16.8\%$) , $\gamma^{20} = 0.70^{20} \approx 0.000798$ ($\approx 0.08\%$)

$\gamma = 0.95$: $\gamma^5 = 0.95^5 \approx 0.77378$ ($\approx 77.4\%$) , $\gamma^{20} = 0.95^{20} \approx 0.35849$ ($\approx 35.8\%$)

At $\gamma = 0.95$, a reward 20 steps in the future still retains almost 36% of its original weight, versus essentially 0% at $\gamma = 0.40$ - this large numeric gap, computed directly from the given discount factors, explains why only $\gamma = 0.95$ lets distant rewards meaningfully influence current decisions.

5. Conclusion

$\gamma = 0.95$ is the most effective discount factor for SARSA in this task because it gives the agent a long effective planning horizon (as the γ^k calculation shows) while remaining below 1 for numerical stability, allowing it to properly value and pursue delayed, larger rewards rather than settling for smaller immediate ones.

10. An RL agent trained in a stochastic environment shows fluctuating rewards in early episodes but stabilizes after 500 episodes. Examine the reward trend and defend how stability indicates policy improvement and robustness of learning.

(5 Marks)

Answer:

1. Interpretation of the Data

In the early episodes of training in a stochastic environment, rewards fluctuate noticeably; after roughly 500 episodes, the reward trend stabilises, settling around a consistent level rather than swinging widely from episode to episode. No specific reward figures are given, so the explanation below is developed from the underlying statistical and RL principles rather than invented numbers.

2. Analysis - Source of Early Fluctuation and Later Stability

Two sources of variability act together in early training: (a) the agent's policy is still immature, with frequent exploratory/near-random actions and inaccurate value estimates, and (b) the environment itself is stochastic, so identical state-action pairs can produce different next states or rewards purely by chance.

As training progresses, each Bellman/TD update effectively averages the received rewards across many samples, so the value estimates increasingly reflect true expected returns rather than single noisy outcomes. The resulting policy becomes less sensitive to individual random events and settles into consistent behaviour.

3. Application of RL Concept - Convergence and Robustness

Stabilisation of reward is a practical, empirical signature of the value function (and hence the greedy policy derived from it) converging: once $Q(s,a)$ estimates stop changing significantly between episodes, the policy stops changing too, and reward variance narrows around its true expected value under that fixed policy.

Robustness follows directly from this: a policy that keeps producing a similar, high reward despite the environment's inherent randomness is one that performs well on average across the full distribution of stochastic outcomes, not one that got lucky in a handful of episodes.

4. Supporting Reasoning (statistical principle, since no numeric reward data is given)

By the law of large numbers, for a sequence of noisy but identically distributed reward samples, the variance of their running average shrinks proportionally to $1/N$ as the number of samples N grows (here, N = number of episodes).

This is why fluctuations that dominate the first few dozen episodes become progressively smaller as episodes accumulate toward 500, without needing invented figures to illustrate the effect - it follows directly from the statistics of repeated sampling under a stochastic, but eventually fixed, policy.

5. Conclusion

The shift from fluctuating to stable rewards indicates that the agent's policy has converged and become robust to the environment's stochastic noise, supporting the conclusion that training beyond 500 episodes has produced a reliable, near-optimal policy for this stochastic environment.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data

Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

